6. HotStuff 네트워킹 및 통신

6.1 네트워킹 아키텍처 개요

6.1.1 통신 모델

HotStuff는 다음과 같은 통신 패턴을 사용합니다:

- **리더-팔로워**: 리더가 제안하고 팔로워들이 응답
- **브로드캐스트**: 모든 복제본에게 메시지 전송
- **포인트-투-포인트**: 특정 복제본에게 직접 전송
- **수집**: 리더가 팔로워들의 응답 수집

6.1.2 네트워크 스택

```
┌─────────────────────────────────────┐
│ Application Layer │
│ (HotStuff Protocol Messages) │
├─────────────────────────────────────┤
│ gRPC Layer │
│ (Serialization/RPC) │
├─────────────────────────────────────┤
│ Gorums Layer │
│ (Quorum-based Communication) │
```

```
├──────────────────────────────────┤
| TCP/TLS Layer |
| (Reliable Transport) |
└──────────────────────────────────┘
```

6.2 Gorums 기반 통신

6.2.1 Gorums 개요

Gorums는 쿼럼 기반 시스템을 위한 gRPC 확장입니다:

- **쿼럼 호출**: 여러 서버에 동시 요청하여 쿼럼 응답 대기
- **구성 관리**: 복제본 집합과 쿼럼 크기 관리
- **장애 허용**: 일부 복제본 장애 시에도 동작
- **성능 최적화**: 병렬 통신과 조기 종료

6.2.2 서비스 정의 (Protocol Buffers)

```
service Hotstuff {
// 블록 제안
rpc Propose(Proposal) returns (google.protobuf.Empty);
// 투표 전송
rpc Vote(PartialCert) returns (google.protobuf.Empty);
// 새 뷰 알림
rpc NewView(SyncInfo) returns (google.protobuf.Empty);
// 타임아웃 전송
```

```
rpc Timeout(TimeoutMsg) returns (google.protobuf.Empty);

// 블록 가져오기

rpc Fetch(BlockHash) returns (Block);

}
```

6.3 서버 구현 (Backend Server)

6.3.1 서버 구조체

```
type Server struct {

// 의존성 모듈들

blockChain modules.BlockChain

configuration modules.Configuration

eventLoop *eventloop.EventLoop

logger logging.Logger

opts *modules.Options

// 서버 설정

id hotstuff.ID

lm latency.Matrix // 지연 시간 시뮬레이션

gorumsSrv *gorums.Server // Gorums 서버

}
```

6.3.2 서버 초기화

```go
func NewServer(opts ...ServerOptions) *Server {
options := &backendOptions{}
for _, opt := range opts {
opt(options)
}
srv := &Server{
id: options.id,
lm: options.latencyMatrix,
}
// 연결 콜백 설정
options.gorumsSrvOpts = append(options.gorumsSrvOpts,
gorums.WithConnectCallback(func(ctx context.Context) {
srv.eventLoop.AddEvent(replicaConnected{ctx})
}))
// Gorums 서버 생성
srv.gorumsSrv = gorums.NewServer(options.gorumsSrvOpts...)
// 서비스 등록
hotstuffpb.RegisterHotstuffServer(srv.gorumsSrv, &serviceImpl{srv})
return srv
}
```

6.3.3 서버 시작

```go
func (srv *Server) Start(addr  string) error {
// TCP 리스너 생성
lis, err := net.Listen("tcp", addr)
if  err != nil {
return  fmt.Errorf("failed to listen on %s: %w", addr, err)
}
// 서버 시작
srv.StartOnListener(lis)
return  nil
}

func (srv *Server) StartOnListener(listener net.Listener) {
go  func() {
err := srv.gorumsSrv.Serve(listener)
if  err != nil {
srv.logger.Errorf("An error occurred while serving: %v", err)
}
}()
}
```

6.4 메시지 처리 구현

6.4.1 제안 메시지 처리

```go
func (impl *serviceImpl) Propose(ctx gorums.ServerCtx, proposal *hotstuffpb.Prop
// 송신자 ID 추출
id, err := GetPeerIDFromContext(ctx, impl.srv.configuration)
if  err != nil {
impl.srv.logger.Warnf("Could not get replica ID: %v", err)
return
}
// 트리 구조 사용 시 제안자 ID 조정
if  impl.srv.opts.ShouldUseTree() {
id = hotstuff.ID(proposal.Block.Proposer)
}
proposal.Block.Proposer = uint32(id)
// 프로토콜 메시지로 변환
proposeMsg := hotstuffpb.ProposalFromProto(proposal)
proposeMsg.ID = id
// 네트워크 지연 시뮬레이션
impl.srv.addNetworkDelay(id)
// 이벤트 루프에 추가
impl.srv.eventLoop.AddEvent(proposeMsg)
}
```

6.4.2 투표 메시지 처리

```go
func (impl *serviceImpl) Vote(ctx gorums.ServerCtx, cert *hotstuffpb.PartialCert

// 송신자 ID 추출

id, err := GetPeerIDFromContext(ctx, impl.srv.configuration)

if  err != nil {

impl.srv.logger.Warnf("Could not get replica ID: %v", err)

return

}

// 네트워크 지연 추가

impl.srv.addNetworkDelay(id)

// 투표 메시지 이벤트 생성

impl.srv.eventLoop.AddEvent(hotstuff.VoteMsg{

ID: id,

PartialCert: hotstuffpb.PartialCertFromProto(cert),

})

}
```

6.4.3 블록 가져오기 처리

```go
func (impl *serviceImpl) Fetch(_ gorums.ServerCtx, pb *hotstuffpb.BlockHash) (*h

// 해시 추출

var  hash hotstuff.Hash

copy(hash[:], pb.GetHash())
```

```go
// 로컬 블록체인에서 블록 검색
block, ok := impl.srv.blockChain.LocalGet(hash)
if !ok {
return  nil, status.Errorf(codes.NotFound, "requested block was not found")
}
impl.srv.logger.Debugf("OnFetch: %.8s", hash)
// 프로토콜 버퍼로 변환하여 반환
return  hotstuffpb.BlockToProto(block), nil
}
```

6.5 클라이언트 구현 (Configuration)

6.5.1 Configuration 구조체

```go
type Config struct {
// Gorums 관리자
mgr *gorums.Manager
// 복제본 정보
replicas  map[hotstuff.ID]*Replica
nodes []*gorums.Node
// 쿼럼 설정
qspec QuorumSpec
// 네트워크 설정
```

```
creds credentials.TransportCredentials
}
```

6.5.2 복제본 연결

```
func (cfg *Config) Connect(replicas []ReplicaInfo) error {
// 노드 정보 생성
var nodeMap = make(map[string]*gorums.Node)
for _, replica := range replicas {
// gRPC 연결 옵션 설정
opts := []grpc.DialOption{
grpc.WithBlock(),
grpc.WithTimeout(time.Second * 10),
}
if cfg.creds != nil {
opts = append(opts, grpc.WithTransportCredentials(cfg.creds))
} else {
opts = append(opts, grpc.WithInsecure())
}
// 메타데이터 추가 (ID 정보)
opts = append(opts, grpc.WithUnaryInterceptor(
func(ctx context.Context, method string, req, reply interface{},
cc *grpc.ClientConn, invoker grpc.UnaryInvoker, opts ...grpc.CallOption) error {
```

```go
ctx = metadata.AppendToOutgoingContext(ctx, "id", strconv.Itoa(int(replica.ID)))
return invoker(ctx, method, req, reply, cc, opts...)
}))
// 노드 생성
node, err := cfg.mgr.NewNode(replica.Address, opts...)
if err != nil {
return fmt.Errorf("failed to connect to replica %d: %w", replica.ID, err)
}
nodeMap[replica.Address] = node
cfg.replicas[replica.ID] = &Replica{
node: node,
id: replica.ID,
}
}
// 노드 리스트 생성
cfg.nodes = make([]*gorums.Node, 0, len(nodeMap))
for _, node := range nodeMap {
cfg.nodes = append(cfg.nodes, node)
}
return nil
}
```

6.5.3 쿼럼 사양 정의

```go
type QuorumSpec struct {
n int // 전체 복제본 수
q int // 쿼럼 크기
}

func (qs QuorumSpec) VoteQF(replies map[uint32]*hotstuffpb.Empty) (*hotstuffpb.
// 쿼럼 크기만큼 응답이 오면 성공
if len(replies) >= qs.q {
return &hotstuffpb.Empty{}, true
}
return nil, false
}

func (qs QuorumSpec) FetchQF(replies map[uint32]*hotstuffpb.Block) (*hotstuffpb
// 첫 번째 유효한 응답 반환
for _, block := range replies {
if block != nil {
return block, true
}
}
return nil, false
```

```
}
```

6.6 메시지 전송 구현

6.6.1 제안 브로드캐스트

```
func (cfg *Config) Propose(proposal hotstuff.ProposeMsg) {
// 프로토콜 버퍼로 변환
pbProposal := hotstuffpb.ProposalToProto(proposal)
// 모든 복제본에게 비동기 전송
ctx, cancel := context.WithTimeout(context.Background(), time.Second*5)
defer  cancel()
// Gorums 쿼럼 호출 (응답 불필요)
cfg.qspec.Propose(ctx, pbProposal)
}
```

6.6.2 투표 전송

```
func (r *Replica) Vote(cert hotstuff.PartialCert) {
// 프로토콜 버퍼로 변환
pbCert := hotstuffpb.PartialCertToProto(cert)
// 특정 복제본에게 전송
ctx, cancel := context.WithTimeout(context.Background(), time.Second*2)
```

```
defer cancel()
_, err := r.node.Vote(ctx, pbCert)
if err != nil {
// 로그 기록 (에러는 무시)
log.Debugf("Failed to send vote to replica %d: %v", r.id, err)
}
}
```

6.6.3 블록 가져오기

```
func (cfg *Config) Fetch(ctx context.Context, hash hotstuff.Hash) (*hotstuff.Blo
// 해시를 프로토콜 버퍼로 변환
pbHash := &hotstuffpb.BlockHash{Hash: hash[:]}
// 쿼럼 호출로 블록 요청
pbBlock, err := cfg.qspec.Fetch(ctx, pbHash)
if err != nil {
return nil, false
}
// 프로토콜 버퍼에서 블록으로 변환
block := hotstuffpb.BlockFromProto(pbBlock)
return block, true
}
```

6.7 네트워크 지연 시뮬레이션

6.7.1 지연 매트릭스

```
type Matrix struct {
enabled  bool
locations []string
latencies [][]time.Duration
idToIndex  map[hotstuff.ID]int
}

func (lm *Matrix) Latency(from, to hotstuff.ID) time.Duration {
if !lm.enabled {
return  0
}
fromIdx, fromOk := lm.idToIndex[from]
toIdx, toOk := lm.idToIndex[to]
if !fromOk || !toOk {
return  0
}
return  lm.latencies[fromIdx][toIdx]
}
```

6.7.2 지연 추가

```go
func (srv *Server) addNetworkDelay(sender hotstuff.ID) {
if !srv.lm.Enabled() {
return
}
delay := srv.lm.Latency(srv.id, sender)
srv.logger.Debugf("Delay between %s and %s: %v",
srv.lm.Location(srv.id), srv.lm.Location(sender), delay)
// 실제 지연 적용
srv.lm.Delay(srv.id, sender)
}

func (lm *Matrix) Delay(from, to hotstuff.ID) {
delay := lm.Latency(from, to)
if  delay > 0 {
time.Sleep(delay)
}
}
```

6.8 TLS 보안 통신

6.8.1 TLS 설정

```go
func  setupTLS(certFile, keyFile, caFile  string) (credentials.TransportCredenti
```

```go
// 인증서 로드
cert, err := tls.LoadX509KeyPair(certFile, keyFile)
if err != nil {
return nil, err
}
// CA 인증서 로드
caCert, err := ioutil.ReadFile(caFile)
if err != nil {
return nil, err
}
caCertPool := x509.NewCertPool()
caCertPool.AppendCertsFromPEM(caCert)
// TLS 설정
tlsConfig := &tls.Config{
Certificates: []tls.Certificate{cert},
RootCAs: caCertPool,
ClientAuth: tls.RequireAndVerifyClientCert,
ClientCAs: caCertPool,
}
return credentials.NewTLS(tlsConfig), nil
}
```

6.8.2 피어 ID 추출

```
func GetPeerIDFromContext(ctx context.Context, cfg modules.Configuration) (hots
peerInfo, ok := peer.FromContext(ctx)
if !ok {
return 0, fmt.Errorf("peerInfo not available")
}
// TLS 인증서에서 ID 추출
if peerInfo.AuthInfo != nil && peerInfo.AuthInfo.AuthType() == "tls" {
tlsInfo, ok := peerInfo.AuthInfo.(credentials.TLSInfo)
if !ok {
return 0, fmt.Errorf("authInfo of wrong type: %T", peerInfo.AuthInfo)
}
if len(tlsInfo.State.PeerCertificates) > 0 {
cert := tlsInfo.State.PeerCertificates[0]
for replicaID := range cfg.Replicas() {
if subject, err := strconv.Atoi(cert.Subject.CommonName); err == nil {
if hotstuff.ID(subject) == replicaID {
return replicaID, nil
}
}
}
}
```

```go
return 0, fmt.Errorf("could not find matching certificate")
}
// 메타데이터에서 ID 추출 (비TLS)
md, ok := metadata.FromIncomingContext(ctx)
if !ok {
return 0, fmt.Errorf("metadata not available")
}
v := md.Get("id")
if len(v) < 1 {
return 0, fmt.Errorf("id field not present")
}
id, err := strconv.Atoi(v[0])
if err != nil {
return 0, fmt.Errorf("cannot parse ID field: %w", err)
}
return hotstuff.ID(id), nil
}
```

6.9 연결 관리

6.9.1 연결 상태 모니터링

```go
type connectionMonitor struct {
```

```go
cfg *Config
logger logging.Logger
// 연결 상태
connections  map[hotstuff.ID]bool
mutex sync.RWMutex
}


func (cm *connectionMonitor) checkConnections() {
cm.mutex.Lock()
defer  cm.mutex.Unlock()
for  id, replica := range  cm.cfg.replicas {
// 연결 상태 확인
state := replica.node.State()
connected := state == connectivity.Ready || state == connectivity.Idle
if  cm.connections[id] != connected {
cm.connections[id] = connected
if  connected {
cm.logger.Infof("Replica %d connected", id)
} else {
cm.logger.Warnf("Replica %d disconnected", id)
}
}
}
```

```
}
```

6.9.2 재연결 로직

```
func (cfg *Config) reconnect(replicaID hotstuff.ID) error {
replica, exists := cfg.replicas[replicaID]
if !exists {
return fmt.Errorf("replica %d not found", replicaID)
}
// 기존 연결 종료
replica.node.Close()
// 새 연결 생성
node, err := cfg.mgr.NewNode(replica.address, replica.dialOpts...)
if err != nil {
return fmt.Errorf("failed to reconnect to replica %d: %w", replicaID, err)
}
replica.node = node
return nil
}
```

6.10 성능 최적화

6.10.1 연결 풀링

```go
type connectionPool struct {
pools  map[string]*grpc.ClientConn
mutex sync.RWMutex
}


func (cp *connectionPool) getConnection(address  string, opts ...grpc.DialOption
cp.mutex.RLock()
if  conn, exists := cp.pools[address];  exists {
cp.mutex.RUnlock()
return  conn, nil
}
cp.mutex.RUnlock()
cp.mutex.Lock()
defer  cp.mutex.Unlock()
// 다시 확인 (double-checked locking)
if  conn, exists := cp.pools[address];  exists {
return  conn, nil
}
// 새 연결 생성
conn, err := grpc.Dial(address, opts...)
if  err != nil {
return  nil, err
```

```
}
cp.pools[address] = conn
return conn, nil
}
```

6.10.2 메시지 배치 처리

```
type messageBatcher struct {
messages []interface{}
timer *time.Timer
mutex sync.Mutex
batchSize int
batchTimeout time.Duration
sendFunc func([]interface{})
}

func (mb *messageBatcher) addMessage(msg interface{}) {
mb.mutex.Lock()
defer mb.mutex.Unlock()
mb.messages = append(mb.messages, msg)
// 배치 크기 도달 시 즉시 전송
if len(mb.messages) >= mb.batchSize {
mb.flush()
```

```go
return
}
// 첫 번째 메시지면 타이머 시작
if len(mb.messages) == 1 {
mb.timer = time.AfterFunc(mb.batchTimeout, func() {
mb.mutex.Lock()
mb.flush()
mb.mutex.Unlock()
})
}
}

func (mb *messageBatcher) flush() {
if len(mb.messages) == 0 {
return
}
if mb.timer != nil {
mb.timer.Stop()
mb.timer = nil
}
mb.sendFunc(mb.messages)
mb.messages = mb.messages[:0] // 슬라이스 재사용
```

```
}
```

이러한 네트워킹 구현을 통해 HotStuff는 분산 환경에서 효율적이고 안전한 통신을 수행하며, 다양한 네트워크 조건에서도 안정적으로 동작할 수 있습니다.